LAB ASSIGNMENT-4

1. Create a class named ***'Rectangle'*** with two data members- *length* and *breadth* and a function to calculate the area which is 'length*breadth'. The class has *three constructors* which are:
   (a) having no parameter - values of both length and breadth are assigned zero.
   (b) having two numbers as parameters - the two numbers are assigned as length and breadth respectively.
   (c) having one number as parameter - both length and breadth are assigned that number.

   Now, create objects of the 'Rectangle' class having none, one and two parameters and print their areas.

Code:

```
#include <iostream>

using namespace std;

class Rectangle

{

private:

   int length, breadth;

public:

   Rectangle()

   {

      length = 0;

      breadth = 0;

   }

   Rectangle(int x)

   {

      length = x;

      breadth = x;

   }

   Rectangle(int l, int b)

   {

      length = l;

      breadth = b;

   }

   void area()

   {

      cout << "Area = " << length * breadth << endl;
```

```
    }
};
int main()
{
    Rectangle r1;
    Rectangle r2(5);
    Rectangle r3(4, 6);
    cout << "Rectangle 1"<<endl;
    r1.area();
    cout << "Rectangle 2”<<endl;
    r2.area();
    cout << "Rectangle 3”<endl;
    r3.area();
    return 0;
}
```

2. Redefine the above program by creating an array of objects of the class Rectangle and calculate area for each object calling different constructors. Also implement constructors with default arguments and destructor in this program.

Code:

```
#include <iostream>
using namespace std;
class Rectangle
{
private:
    int length, breadth;
public:
    Rectangle(int l = 0, int b = 0)
    {
        length = l;
        breadth = b;
    }
    void area()
    {
        cout << "Area = " << length * breadth << endl;
    }

    ~Rectangle()
    {
        cout << "Destructor Called";
```

```
    }
};
int main()
{
    Rectangle r[3] = {Rectangle(), Rectangle(5), Rectangle(4,6)};
    for(int i = 0; i < 3; i++)
    {
        cout << "Rectangle " << i + 1 << ": ";
        r[i].area();
    }
    return 0;
}
```

3. Verify the following about *destructor* by writing the program:
   (i) Name should begin with tilde sign(~) and must match class name.
   (ii) There cannot be more than one destructor in a class.
   (iii) Destructors do not allow any parameter.
   (iv) They do not have any return type, just like constructors.

Code:

```
#include <iostream>

using namespace std;

class Demo

{

public:

   Demo()

   {

      cout << "Constructor Called"<<endl;

   }

   ~Demo()

   {

      cout << "Destructor Called"<<endl;

   }

};


int main()

{

   Demo d;
```

```
    return 0;
}
```

4. Implement dynamic memory allocation. Use *new* and *delete* keywords. (For integer variable, float variable, integer array, float array, class objects, Array of Objects)

Code:

```
#include <iostream>
using namespace std;
class Test
{
public:
    void display()
    {
        cout << "Object Created";
    }
};
int main()
{
    int *p = new int;
    *p = 10;
    cout << "Integer Value = " << *p << endl;
    float *f = new float;
    *f = 5.5;
    cout << "Float Value = " << *f << endl;
    int *arr = new int[3];
    arr[0] = 1;
    arr[1] = 2;
    arr[2] = 3;
    cout << "Integer Array"<<endl;

    for(int i = 0; i < 3; i++)
    {
        cout << arr[i] << " ";
```

```
    }
    float *farr = new float[3];
    farr[0] = 1.1;
    farr[1] = 2.2;
    farr[2] = 3.3;
    cout << "Float Array"<<endl;
    for(int i = 0; i < 3; i++)
    {
        cout << farr[i] << " ";
    }
    Test *obj = new Test;
    obj->display();
    Test *objArr = new Test[2];
    cout << "Array of Objects Created"<<endl;
    delete p;
    delete f;
    delete[] arr;
    delete[] farr;
    delete obj;
    delete[] objArr;
    return 0;
}
```